Table des matières

Le poste de travail du parcours — guide d'installation 1
Comment ce guide fonctionne 2
Les trois terminaux, et ce que chacun sert 3
Ce que vous installez, et pourquoi 4
Les versions de référence 5
Étape 1 — Installer Python, avec conda 6
Étape 2 — Installer Git 6
Étape 3 — Installer DuckDB 7
Étape 4 — Installer Ubuntu dans WSL 2, puis Docker Engine 7
Étape 5 — Régler l'arrêt automatique d'Ubuntu 9
Étape 6 — Installer le pilote ODBC de PostgreSQL 10
Étape 7 — Installer PostgreSQL sur le poste 10
Récapitulatif 11
Dépannage 12
Annexe — Tout le code, d'un seul tenant 14

Le poste de travail du parcours — guide d'installation

Commun aux sept formations du domaine des données et à ses fondations

Ce que vous allez faire. Monter une fois le poste que tout le parcours emploie : Python et son environnement, Git, DuckDB, Docker dans l'Ubuntu de WSL 2 et le pilote ODBC de PostgreSQL. Les données de Boréal n'en font pas partie : elles se génèrent au Cours 00 de Boréal, comme chaque cas fait les siennes à son premier pas. Vous n'installerez plus rien d'un bloc ensuite : chaque cours qui allume un service le fait au moment où il s'en sert, en quelques commandes, et ce guide reste la référence à laquelle il renvoie.

Ce guide explique chaque geste et ce qu'il prouve. Pour enchaîner les commandes sans les explications, suivez `Labo_installation.md`, dans le même dossier : les mêmes sept étapes, dans le même ordre et sous les mêmes numéros, chacune avec un renvoi à sa section ici. Revenez à ce guide quand une sortie ne ressemble pas à l'attendu, et à son dépannage, en fin de document.

La liste de contrôle du socle :

© 2026 iAlexMG. Tous droits réservés.

- ☐ Environnement conda `formations` actif ; `import duckdb, pyodbc, pytest` qui répond
- ☐ `git --version` qui répond, `user.name` et `user.email` réglés
- ☐ `duckdb --version` qui répond dans un terminal Windows
- ☐ Ubuntu dans WSL 2, Docker Engine installé, `docker run hello-world` qui affiche `Hello from Docker!`
- ☐ Pilote psqlODBC installé : `PostgreSQL Unicode(x64)` dans la liste des pilotes ODBC

Comment ce guide fonctionne

Le parcours partage une seule plateforme de données et un seul poste de travail. Ce guide les monte une fois, pour les sept formations et les fondations. Rien ici n'est le travail d'un métier de la donnée : en entreprise, les TI montent le poste, l'équipe des systèmes livre la plateforme, et les systèmes sources tournent déjà. On le fait ici parce qu'il faut bien une machine à qui parler.

Ce que ce guide monte, on l'appelle le socle : ce que presque tout le parcours emploie. Ce qu'il ne monte pas, ce sont les services qu'une seule formation allume, et qui coûtent cher à laisser dormir : la base de l'ERP de Boréal, Airflow, Spark, Kafka, une base de séries temporelles. Chacun s'installe au début du cours qui s'en sert, dans un bloc d'ouverture de quelques commandes qui renvoie ici pour le reste.

Pourquoi — ne pas tout monter d'avance. Monter toute la pile avant le premier cours met trois à cinq heures de téléchargement et de configuration entre vous et votre premier geste de métier, et vous fait installer au premier jour des services que le neuvième cours vous expliquera. Vous les auriez oubliés avant de vous en servir. La règle du parcours est l'inverse : le socle maintenant, l'outil le jour où il sert.

Étape	Ce qu'elle installe	Qui en a besoin
1	Python, par conda	tout le parcours
2	Git	tout le parcours
3	DuckDB, le paquet et la ligne de commande	tout le parcours
4	Ubuntu dans WSL 2, puis Docker Engine	tout le parcours, sauf Architecture et Gouvernance
5	le réglage d'arrêt automatique d'Ubuntu	avec l'étape 4
6	le pilote ODBC de PostgreSQL	Data Engineering
7	PostgreSQL sur le poste	la fondation SQL avancé

© 2026 iAlexMG. Tous droits réservés.

Les étapes 6 et 7 se sautent tant que la formation qui les demande n'est pas commencée. Les autres se font dans l'ordre : chacune vérifie la précédente.

Ce que chaque formation ajoute au socle, et où :

Formation	Ce qu'elle allume en plus	Où
Data Engineering	le projet `Projet_Boreal_intermediaire/` et ses sources, puis la base de l'ERP de Boréal, Airflow, Spark, Kafka, TimescaleDB, Grafana	le projet et ses sources au Cours 00, le reste au bloc d'ouverture du cours qui s'en sert
Analytics Engineering	dbt sur DuckDB	à son premier cours
Business Intelligence	Power BI Desktop	au cours qui construit le rapport
Data Science, ML Engineering	scikit-learn, MLflow	à leur premier cours
AI Engineering	Ollama et un modèle local	à son premier cours
Architecture et Gouvernance	rien : de quoi écrire et dessiner	sans objet
SQL avancé	PostgreSQL et AdventureWorks	étape 7 de ce guide, puis son Cours 00

Le `README.md` du parcours dit ce que chacun de ces outils coûte et ce qu'on perd à le sauter.

Les trois terminaux, et ce que chacun sert

Le parcours fait travailler dans trois terminaux. Un cours dit toujours dans lequel vous êtes ; ce qui suit dit une fois pour toutes pourquoi il y en a trois, pour que vous n'ayez plus à changer de fenêtre sur ordre sans comprendre.

Terminal	Ce qu'on y tape	Comment on l'ouvre
l'Anaconda Prompt	Python, `pip`, Git, les générateurs, `psql`	menu Démarrer, *Anaconda Prompt*
la ligne de commande DuckDB	le SQL de vérification	`duckdb` dans l'Anaconda Prompt
Ubuntu, dans WSL 2	tout ce qui touche Docker	`wsl` dans l'Anaconda Prompt, ou *Ubuntu* au menu Démarrer

La raison de fond tient en une phrase : Docker Engine vit dans Ubuntu, et pas sous Windows. Tout ce qui parle à Docker (les services en conteneurs, Airflow, Kafka) se tape donc dans Ubuntu, et le reste dans l'Anaconda Prompt. La ligne de commande DuckDB, elle, n'est pas un terminal du système : c'est un client SQL qui s'ouvre dans le terminal où vous êtes, et qui attend du SQL. On en sort par `.quit`.

© 2026 iAlexMG. Tous droits réservés.

Ce que vous installez, et pourquoi

Pièce	Son rôle	Pourquoi celle-là
conda, environnement `formations`	isoler Python et ses paquets	le même environnement sert toutes les formations de l'atelier ; on le crée une fois
Git	versionner le code	un projet qu'on ne peut pas reconstruire depuis son dépôt n'est pas un livrable, et chaque formation en ouvre un
DuckDB	interroger des fichiers en SQL, sans serveur	il tient sur un portable, lit CSV, JSON et Parquet sur place, et se charge comme une bibliothèque Python
WSL 2 et Docker	faire tourner les services dans des conteneurs Linux	installer ces services directement sous Windows coûte des heures sans rien apprendre, et ni Airflow ni Kafka n'y tournent ; un conteneur se jette et se recrée en une commande
psqlODBC	laisser un programme Windows lire une base PostgreSQL	Data Engineering lit la base de l'ERP comme on lit une base de production, par un pilote
PostgreSQL	un moteur transactionnel sur le poste	la fondation SQL avancé y travaille son second terrain, celui des bases qui font tourner l'entreprise

WSL 2 (*Windows Subsystem for Linux*) fait tourner un vrai Linux, ici Ubuntu, à côté de Windows, sans machine virtuelle à gérer soi-même. Docker Engine s'y installe comme sur un serveur, avec les commandes de sa documentation officielle : c'est la voie du parcours. Depuis Ubuntu, les fichiers de Windows restent visibles sous `/mnt/c/`.

Pourquoi — Docker Engine plutôt que Docker Desktop. Docker Desktop, l'application graphique de Docker, fait le même travail sur Windows 11 et sur macOS, et les commandes des labos y sont identiques. Le parcours ne le prend pas comme référence : Docker ne le soutient plus sur Windows 10, sorti du calendrier de service de Microsoft en octobre 2025, et sa gratuité est réservée aux particuliers et aux petites entreprises. Docker Engine, dans Ubuntu, est gratuit sans condition et se comporte comme sur le serveur Linux où vos conteneurs finiront.

Tout se range dans le dossier du parcours :

```
Data_Domain/                    le dossier du parcours, dans le dépôt des formations
├── 00_Installation/            ce guide, et son labo
├── _univers/                   ce que partagent les formations du parcours
```


© 2026 iAlexMG. Tous droits réservés.

```
│   ├── boreal/generer_boreal.py    le générateur des données de Boréal
│   └── plateforme/                 les fichiers des services, reçus tout prêts
├── 01_Data_Engineering/            les supports d'une formation
└── Projet_Boreal_intermediaire/    votre projet, créé au Cours 00 de Boréal et ignoré par le dépôt
    ├── sources/                    ce que les systèmes de Boréal livrent, générés au même Cours 00
    ├── plateforme/                 les services, quand un cours en allume un
    └── 01_de/                      votre dépôt Git, créé au premier cours de Data Engineering
```

Les versions de référence

Les chiffres et les sorties que les cours affichent ont été pris avec ces versions, relevées le 2026-09-17. Une version plus récente convient presque toujours ; si une sortie diffère de ce qu'un cours annonce, c'est la première chose à regarder. Les `requirements.txt` des formations bornent chaque paquet sous sa prochaine version majeure, `duckdb>=1.5,<2` : une version majeure peut changer ce qu'une commande affiche, et remonter une borne demande de remesurer les chiffres des cours.

Outil	Version de référence	Relevé le
Python	3.11	2026-09-17
DuckDB	1.5.5, publiée le 2026-07-22	2026-09-17
`pyodbc` et `pytest`	5.3.0 et 9.1.1	2026-09-22
`dbt-core` et `dbt-duckdb`	1.12.3 et 1.11.0	2026-09-22
Ubuntu	26.04 LTS ; la 24.04 convient aussi	2026-09-17
psqlODBC	`REL-18_00_0004`	2026-09-17
PostgreSQL, sur le poste	18	2026-09-17
PostgreSQL, en conteneur	image `postgres:18` pour l'ERP de Boréal, `postgres:16` pour la base de métadonnées d'Airflow	2026-09-17
Airflow	3.3.1, publiée le 2026-08-12	2026-09-17
TimescaleDB, en conteneur	image `timescale/timescaledb:2.30.1-pg17`	2026-09-20
Le courtier MQTT et Kafka, en conteneur	images `eclipse-mosquitto:2.1.2-alpine` et `apache/kafka:4.3.1`	2026-09-23

© 2026 iAlexMG. Tous droits réservés.

Outil	Version de référence	Relevé le
`paho-mqtt`, `confluent-kafka` et `psycopg`	2.1.0, 2.15.1 et 3.3.6	2026-09-23
PyIceberg, `pyiceberg-core` et `deltalake`	0.12.0, 0.10.1 et 1.6.5	2026-09-23
PyYAML	6.0.3	2026-09-23
Grafana, en conteneur	image `grafana/grafana:13.2.2`	2026-09-23

Étape 1 — Installer Python, avec conda

Sauf mention contraire, l'Anaconda Prompt de ce guide travaille depuis `Data_Domain`.

L'atelier utilise un seul environnement conda, `formations`, partagé par toutes ses formations. Si vous l'avez déjà, activez-le. Sinon, créez-le une fois :

```
conda create -n formations python=3.11 -y
conda activate formations
```

L'invite affiche `(formations)` au début de la ligne. C'est le signe que vous êtes dans le bon environnement, et chaque formation vous demandera de le vérifier.

Les paquets Python d'une formation tiennent dans son fichier `requirements.txt`. Celui de Data Engineering installe DuckDB, `pyodbc` qui lit la base de l'ERP par ODBC, `pytest` qui lance les tests du pipeline, les trois clients du chemin chaud de son Cours 09, pour le courtier MQTT, pour Kafka et pour PostgreSQL, les deux bibliothèques des tables ouvertes de son Cours 10, PyIceberg et `deltalake`, et PyYAML, qui lit les contrats de données de son Cours 11. Installez-les maintenant si vous commencez par cette formation :

```
pip install -r 01_Data_Engineering\requirements.txt
python -c "import duckdb, pyodbc, pytest; print(duckdb.__version__)"
```

La seconde ligne ne doit rien afficher d'autre qu'un numéro de version : une erreur `ModuleNotFoundError` voudrait dire qu'un paquet manque, ou que `formations` n'est pas actif.

Étape 2 — Installer Git

Git versionne votre code. Chaque formation du parcours ouvre un dépôt dès son premier cours, et une formation entière lui est consacrée dans l'atelier.

Dans l'Anaconda Prompt :

```
git --version
```

S'il répond un numéro de version, passez au réglage de l'identité. Sinon, installez-le depuis `https://git-scm.com/download/win` (l'installateur 64 bits), en acceptant les choix par défaut, puis fermez et rouvrez l'Anaconda Prompt avant de revérifier.

© 2026 iAlexMG. Tous droits réservés.

Git refuse d'enregistrer une modification tant qu'il ne sait pas qui la signe. Réglez-le une fois, pour tous vos dépôts :

```
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.com"
git config --global --list
```

La dernière ligne doit montrer vos deux valeurs. Sans elles, le premier `git commit` d'un labo s'arrête sur `Author identity unknown`.

Étape 3 — Installer DuckDB

DuckDB s'installe deux fois, pour deux usages. Le paquet Python, posé à l'étape 1, sert au code qui interroge l'entrepôt. La ligne de commande DuckDB est le client SQL des labos : on y tape les requêtes de vérification. Elle s'installe par winget, le gestionnaire de paquets de Windows :

```
winget install DuckDB.cli
```

winget ajoute la commande `duckdb` au chemin de recherche de Windows, mais seul un terminal ouvert après l'installation la voit. Fermez l'Anaconda Prompt, rouvrez-le, réactivez `formations`, revenez dans `Data_Domain`, puis vérifiez :

```
duckdb --version
```

Python affiche `1.5.5`, la ligne de commande `v1.5.5 (Variegata)` suivi d'un identifiant. DuckDB ne demande ni service, ni port, ni mot de passe. Sous Windows, il a seulement besoin du Microsoft Visual C++ Redistributable, déjà présent sur la plupart des postes.

Étape 4 — Installer Ubuntu dans WSL 2, puis Docker Engine

Docker fait tourner les services du parcours. Il s'installe en trois temps : Ubuntu dans WSL 2, le dépôt de paquets de Docker, puis Docker Engine lui-même.

D'abord, ce que le poste a déjà. Dans l'Anaconda Prompt :

```
wsl -l -v
```

Si la liste montre une distribution Ubuntu en version 2, passez au paragraphe suivant. Sinon, ouvrez un terminal Windows en administrateur et installez-la :

```
wsl --install -d Ubuntu-26.04
```

L'installation peut demander un redémarrage. Au premier lancement, Ubuntu vous demande un nom d'utilisateur et un mot de passe : ce sont ceux de Linux, sans lien avec votre compte Windows, et `sudo` vous redemandera ce mot de passe.

Ouvrez Ubuntu : tapez `wsl` dans l'Anaconda Prompt, ou lancez Ubuntu depuis le menu Démarrer. L'invite change, vous êtes dans Linux. Vérifiez que systemd, le gestionnaire de services de Linux, est bien le premier processus :

```
ps -p 1 -o comm=
```


© 2026 iAlexMG. Tous droits réservés.

La réponse attendue est `systemd`. C'est lui qui démarrera Docker chaque fois qu'Ubuntu démarre. S'il répond autre chose, voyez le dépannage.

Ensuite, Docker Engine, en trois blocs à lancer dans l'ordre. Ces commandes sont celles de la documentation officielle de Docker pour Ubuntu (`docs.docker.com/engine/install/ubuntu`), coupées pour qu'on voie ce que fait chacun.

Premier bloc, la clé qui signe les paquets de Docker. Lancez ces cinq lignes une à une, en attendant que chacune rende l'invite :

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Les deux premières relisent la liste des paquets d'Ubuntu, puis installent `curl`, qui télécharge, et `ca-certificates`, qui lui permet de vérifier un site en HTTPS ; les deux sont souvent déjà là, et `apt` le dit. Les trois suivantes créent le dossier des clés d'`apt`, y téléchargent la clé publique de Docker et la rendent lisible : `apt` s'en servira pour vérifier que chaque paquet vient bien de Docker. `sudo` demande votre mot de passe Linux au premier usage.

Piège — coller les cinq lignes d'un coup. Quand `apt install` pose sa question de confirmation, il lit la réponse dans ce qui a été collé, et prend la ligne suivante pour elle. Une ligne à la fois ; à la question, `Y`.

Deuxième bloc, la déclaration du dépôt de Docker. C'est une seule commande, sur huit lignes : collez-la d'un seul tenant, de `sudo tee` jusqu'à la ligne `EOF` comprise, puis Entrée.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

`<<EOF` passe à `sudo tee` tout ce qui suit, jusqu'à la ligne `EOF`, et `tee` l'écrit dans `/etc/apt/sources.list.d/docker.sources` tout en le réaffichant : voir les six lignes revenir à l'écran est l'attendu. Au passage, les deux `$(…)` se remplacent par le nom de code de votre Ubuntu et par l'architecture du poste. Le fichier dit à `apt` où trouver les paquets de Docker, pour quelle version d'Ubuntu, et avec quelle clé les vérifier.

Troisième bloc, Docker Engine lui-même, depuis ce dépôt. Une ligne à la fois, ici aussi :

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

`apt update` relit les dépôts, celui de Docker compris. `apt install` pose cinq paquets : le moteur, son client en ligne de commande, `containerd`, qui exécute les conteneurs, et les greffons `buildx` et `compose`. Il demande une confirmation : `Y`.

© 2026 iAlexMG. Tous droits réservés.

Dernier réglage : sans lui, chaque commande `docker` demanderait `sudo`. Ajoutez votre utilisateur au groupe `docker`, puis fermez Ubuntu pour que le changement prenne effet :

```
sudo usermod -aG docker $USER
exit
```

Rouvrez Ubuntu (`wsl`), puis :

```
docker --version
docker compose version
docker run hello-world
```

Les deux premières lignes affichent les versions du moteur et de `compose`, l'outil qui démarre plusieurs services d'un coup. La troisième télécharge une image depuis le registre public de Docker, en crée un conteneur et l'exécute : il affiche un texte qui commence par `Hello from Docker!`, puis s'arrête. Ce conteneur arrêté existe encore. Listez-le, puis faites le ménage :

```
docker ps -a
docker container prune
```

`docker ps -a` montre tous les conteneurs, y compris ceux qui sont arrêtés ; `docker container prune` supprime les arrêtés (confirmez par `y`). L'image, elle, reste en cache : un prochain `docker run hello-world` ne la retéléchargera pas. Quittez Ubuntu avec `exit`. Ce que sont une image et un conteneur, et pourquoi on ne modifie jamais un conteneur à la main, s'apprend dans le cours qui vous fera en construire un.

Astuce — rendre la mémoire. Docker démarre avec Ubuntu, par systemd, et WSL peut réserver jusqu'à la moitié de la mémoire de la machine. Quand vous avez fini, `wsl --shutdown`, tapé dans l'Anaconda Prompt, arrête Ubuntu et ses conteneurs d'un coup.

Astuce — Docker Desktop sur Windows 11 ou sur macOS. Si vous avez déjà Docker Desktop, gardez-le : `docker run hello-world` et toutes les commandes des labos s'y tapent pareil, dans votre terminal habituel. Le début de ce guide dit pourquoi le parcours ne le prend pas comme référence.

Piège — Docker Desktop et Docker Engine sur le même Windows. Si Docker Desktop est resté installé sur le poste où vous venez de monter Docker Engine, deux moteurs cohabitent, chacun avec ses images et ses conteneurs. Dans Ubuntu, `docker` parle à Docker Engine ; dans l'Anaconda Prompt, `docker` parle à Docker Desktop. Un conteneur lancé du mauvais côté est invisible de l'autre, et deux conteneurs qui publient le même port se le disputent. Tapez les commandes `docker` des labos dans Ubuntu. En cas de doute, `docker info` le dit à sa ligne `Operating System` : `Ubuntu 26.04 LTS` pour Docker Engine, `Docker Desktop` pour l'autre. Le plus simple reste de désinstaller Docker Desktop (Paramètres de Windows, Applications).

Piège — VirtualBox et WSL 2 sur la même machine. Si vous utilisez VirtualBox (formation Linux), sachez que WSL 2 active la plateforme de virtualisation de Windows. Les versions récentes de VirtualBox savent cohabiter avec elle, mais leurs machines virtuelles peuvent tourner plus lentement. Ce n'est pas une panne.

Étape 5 — Régler l'arrêt automatique d'Ubuntu

© 2026 iAlexMG. Tous droits réservés.

WSL arrête Ubuntu quelques secondes après la fermeture de votre dernier terminal Ubuntu, et les conteneurs s'arrêtent avec elle. En cours de labo, cela veut dire une base qui disparaît parce que vous avez fermé une fenêtre. Le délai se règle dans `%UserProfile%\.wslconfig`, un fichier que Windows ne crée pas tout seul. Dans l'Anaconda Prompt :

```
notepad %UserProfile%\.wslconfig
```

Le Bloc-notes propose de créer le fichier s'il n'existe pas. Écrivez-y ces deux lignes, enregistrez, fermez, puis `wsl --shutdown` pour que le réglage soit relu au prochain démarrage :

```
[general]
instanceIdleTimeout=3600000
```

La valeur est en millisecondes : une heure, au lieu des 15 secondes par défaut. La valeur `-1` existe et ne l'arrête jamais, mais elle garde aussi vos conteneurs allumés jusqu'au redémarrage du poste, ce qui n'est pas ce qu'on veut sur un portable.

Piège — le réglage voisin, qui ne règle pas ce délai. `vmIdleTimeout`, qu'on croise souvent dans les réponses en ligne, arrête la machine virtuelle entière et n'existe que sur Windows 11. C'est `instanceIdleTimeout`, dans la section `[general]`, qui commande l'arrêt d'une distribution inactive. Réglé le 2026-09-17 d'après la documentation de WSL (`learn.microsoft.com`, tableau « General WSL settings ») ; le comportement sous Windows 10 reste à vérifier sur le terrain, et en attendant, gardez une fenêtre Ubuntu ouverte pendant un labo qui fait tourner des conteneurs.

Étape 6 — Installer le pilote ODBC de PostgreSQL

Pour la formation Data Engineering, qui lit la base de l'ERP de Boréal depuis Windows.

Le pilote psqlODBC s'installe une fois, avec les droits d'administrateur. Téléchargez l'installateur 64 bits depuis la page des versions de psqlODBC, `https://www.postgresql.org/ftp/odbc/releases/` : la dernière version est `REL-18_00_0004`, et son fichier `psqlodbc_x64.msi`. Lancez-le et acceptez les choix par défaut.

Puis, dans l'Anaconda Prompt, `formations` actif :

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

La liste des pilotes ODBC du poste doit contenir un pilote dont le nom commence par `PostgreSQL Unicode`. Sur le poste de l'auteur, quatre entrées PostgreSQL apparaissent, deux jeux de caractères pour deux architectures :

```
['PostgreSQL ANSI(x64)', 'PostgreSQL ANSI', 'PostgreSQL Unicode(x64)', 'PostgreSQL Unicode']
```

C'est `PostgreSQL Unicode(x64)` qui sert : Unicode parce que les libellés de l'ERP portent des accents, et 64 bits parce que le Python de `formations` l'est. Si aucune entrée n'apparaît, le pilote n'est pas installé, ou vous avez pris la version 32 bits.

Étape 7 — Installer PostgreSQL sur le poste

© 2026 iAlexMG. Tous droits réservés.

Pour la fondation SQL avancé, qui y monte sa base transactionnelle. Le parcours lui-même n'en a pas besoin : ses bases PostgreSQL tournent en conteneurs.

PostgreSQL s'installe sous Windows par l'installateur d'EDB, `https://www.postgresql.org/download/windows/`. Trois choses à surveiller pendant l'installation :

- laissez le port `5432` ;
- choisissez un mot de passe pour l'utilisateur `postgres`, et notez-le : rien ne le redemandera, et tout le reste en dépend ;
- gardez pgAdmin 4, l'interface graphique, et surtout les outils en ligne de commande.

À la fin, l'installateur propose *Stack Builder*, des extensions facultatives : cliquez sur *Cancel*. PostgreSQL tourne dès lors comme un service Windows, et démarre avec la machine.

L'installateur d'EDB n'ajoute pas les outils en ligne de commande au chemin de recherche de Windows. Ajoutez-les, en remplaçant `18` par votre version, puis fermez et rouvrez l'Anaconda Prompt :

```
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path","User") + ";C:\Program Files\PostgreSQL\18\bin", "User")
```

Cette commande se tape dans PowerShell, pas dans l'Anaconda Prompt : ouvrez *Windows PowerShell* au menu Démarrer, collez-la, puis revenez à l'Anaconda Prompt pour la suite. Vérifiez :

```
psql --version
psql -U postgres -c "SELECT version();"
```

La première répond `psql (PostgreSQL) 18.6`, ou un numéro voisin. La seconde demande le mot de passe de `postgres` : tapez-le, rien ne s'affiche pendant la frappe, c'est normal. Elle répond une longue ligne qui commence par `PostgreSQL 18.6` et décrit la construction du serveur. Si elle répond au lieu de cela `password authentication failed`, c'est le mot de passe choisi à l'installation qui ne concorde pas.

Piège — pgAdmin ne remplace pas le chemin de recherche. pgAdmin se passe de ligne de commande pour écrire des requêtes, et c'est un bon client. Mais le chargement d'un jeu de données passe par `createdb` et `psql`, qui sont des programmes : sans le réglage ci-dessus, ils répondent « n'est pas reconnu », quel que soit le client que vous employez ensuite.

Récapitulatif

Vous avez…	**Preuve**
installé Python	`(formations)` actif ; `import duckdb, pyodbc, pytest` passe

© 2026 iAlexMG. Tous droits réservés.

Vous avez…	Preuve
installé Git	`git --version` répond, et `git config --global --list` montre votre nom et votre courriel
installé DuckDB	`duckdb --version` répond `v1.5.5`, ou une version plus récente
installé Docker	dans Ubuntu, `docker run hello-world` affiche `Hello from Docker!`
réglé l'arrêt d'Ubuntu	`%UserProfile%\.wslconfig` contient `instanceIdleTimeout`
installé le pilote ODBC	`pyodbc.drivers()` contient `PostgreSQL Unicode(x64)`
installé PostgreSQL	`psql -U postgres -c "SELECT version();"` répond `PostgreSQL 18.6…`

Le poste est prêt. Les données de Boréal se génèrent à son Cours 00, qui crée aussi le projet. Ouvrez le `README.md` du parcours pour choisir par où entrer, puis la mise en situation de la formation que vous commencez : elle dit pour qui vous travaillez avant que vous tapiez une ligne. Chaque cours qui a besoin d'un service de plus l'allume lui-même, en tête de son labo, et renvoie ici pour le reste.

Dépannage

Symptôme	Cause probable	Correctif
`'conda' n'est pas reconnu en tant que commande interne`	un terminal ordinaire au lieu de l'Anaconda Prompt	ouvrir l'Anaconda Prompt
`ERROR: Could not open requirements file`	l'Anaconda Prompt n'est pas dans `Data_Domain`	se placer dans `Data_Domain`, puis relancer l'installation des paquets
`ModuleNotFoundError: No module named 'duckdb'` (ou `pyodbc`, `pytest`)	l'environnement `formations` n'est pas actif, ou les paquets n'y sont pas installés	`conda activate formations`, puis l'étape 1
`'git' n'est pas reconnu en tant que commande interne`	Git n'est pas installé, ou le terminal a été ouvert avant l'installation	installer Git, puis fermer et rouvrir l'Anaconda Prompt
`Author identity unknown` au premier commit d'un labo	`user.name` et `user.email` ne sont pas réglés	les deux `git config --global` de l'étape 2

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause probable	Correctif
`'duckdb' n'est pas reconnu en tant que commande interne`	la ligne de commande DuckDB n'est pas installée, ou l'Anaconda Prompt a été ouvert avant son installation	`winget install DuckDB.cli`, puis fermer et rouvrir l'Anaconda Prompt
`wsl --install` refuse de s'exécuter	le terminal n'est pas ouvert en administrateur	rouvrir le terminal en administrateur (clic droit, *Exécuter en tant qu'administrateur*)
L'installation d'Ubuntu échoue avec l'erreur `0x80370102`	la virtualisation matérielle est coupée	l'activer dans le BIOS ou l'UEFI (Intel VT-x ou AMD-V), puis redémarrer
WSL réclame une mise à jour	WSL trop ancien	dans un terminal en administrateur : `wsl --update`, puis redémarrer
`ps -p 1 -o comm=` ne répond pas `systemd`	systemd n'est pas activé dans Ubuntu	dans Ubuntu, ajouter à `/etc/wsl.conf` les deux lignes `[boot]` et `systemd=true` (`sudo nano /etc/wsl.conf`), puis `wsl --shutdown` dans l'Anaconda Prompt, et rouvrir Ubuntu
`E: Unable to locate package docker-ce`	le dépôt de Docker n'est pas déclaré, ou `apt update` n'a pas tourné après	refaire le bloc qui commence par `sudo tee`, puis `sudo apt update`, puis l'installation
`permission denied while trying to connect to the Docker daemon socket`	le groupe `docker` n'est pas encore actif dans ce terminal	`exit`, rouvrir Ubuntu ; si l'erreur reste, `wsl --shutdown` dans l'Anaconda Prompt, puis rouvrir Ubuntu
`Cannot connect to the Docker daemon at unix:///var/run/docker.sock`	le service Docker n'est pas démarré	`sudo systemctl start docker` ; s'il échoue, vérifier systemd (ligne plus haut)
Ubuntu s'arrête dès que vous fermez sa fenêtre, et vos conteneurs avec elle	le délai d'inactivité de WSL est au défaut	l'étape 5 ; en attendant, garder une fenêtre Ubuntu ouverte
`pyodbc.drivers()` ne montre aucun pilote `PostgreSQL`	psqlODBC n'est pas installé, ou c'est la version 32 bits	installer `psqlodbc_x64.msi` (étape 6)
`'psql' n'est pas reconnu en tant que commande interne`	les outils en ligne de commande de PostgreSQL ne sont pas dans le chemin de recherche	la commande PowerShell de l'étape 7, puis rouvrir l'Anaconda Prompt

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause probable	Correctif
`psql: error: connection to server … failed: FATAL: password authentication failed`	le mot de passe de `postgres` n'est pas celui choisi à l'installation	le retrouver ; sinon, réinstaller PostgreSQL, ou changer le mot de passe depuis pgAdmin

---

Annexe — Tout le code, d'un seul tenant

Pour rejouer le socle sur un autre poste sans relire les explications. L'Anaconda Prompt travaille depuis `Data_Domain`.

1 — Python, Git et DuckDB *(Anaconda Prompt)*

```
conda create -n formations python=3.11 -y
conda activate formations
pip install -r 01_Data_Engineering\requirements.txt
python -c "import duckdb, pyodbc, pytest; print(duckdb.__version__)"
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.com"
winget install DuckDB.cli
```

(Après `winget`, fermer et rouvrir l'Anaconda Prompt, réactiver `formations`, puis `duckdb --version`.)

2 — Docker Engine *(dans Ubuntu ; `wsl -l -v`, et au besoin `wsl --install -d Ubuntu-26.04` en administrateur)*

```
ps -p 1 -o comm=

sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```


© 2026 iAlexMG. Tous droits réservés.

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
exit
```

Puis, dans Ubuntu rouvert :

```
docker --version
docker compose version
docker run hello-world
docker ps -a
docker container prune
exit
```

3 — Les pilotes et les moteurs *(installer `psqlodbc_x64.msi` et, pour SQL avancé, PostgreSQL ; puis, dans l'Anaconda Prompt)*

```
python -c "import pyodbc; print(pyodbc.drivers())"
psql --version
```


© 2026 iAlexMG. Tous droits réservés.